

Подделка запроса на стороне сервера (SSRF)

Описание проблемы

Server-side request forgery - это уязвимость, эксплуатация которой позволяет злоумышленнику заставить серверное приложение отправлять запросы к произвольным ресурсам. Этой уязвимости в основном подвержены веб-приложения, которые взаимодействуют со сторонними сервисами либо загружают данные по ссылкам. В таких системах совершаемые запросы бывают подконтрольны пользователям. И в результате подмены ссылки запрос уходит уже на другие ресурсы.

Допустим, git.sbis.ru доступен только из внутренней сети. Но, проэксплуатировав SSRF на online.sbis.ru, злоумышленник может получить доступ к git.sbis.ru, поскольку запрос уже будет идти из внутренней сети компании.

Обычно SSRF делят на слепые, полуслепые, и не слепые. Отличаются они только количеством выводимых в ответ данных - это может быть как полный ответ на запрос, так и разное время ответа или сообщения об ошибках.

Пример небезопасного кода

Обычно мы встречаемся с подобными запросами - где злоумышленником полностью контролируется URL

```
1 url = request.args.get("url")
2
3 http_client = IHTTPClient()
4 response = http_client.Get(url)
```

Также часто можно наткнуться на контроль хоста, на который уходит запрос:

```
1 domain = request.args.get("domain")
2
3 http_client = IHTTPClient()
4 response = http_client.Get(f"https://{domain}/service/ping")
```

А приведённый ниже код может выглядеть безопасным, но через `../` мы можем отправить запрос на любой эндпоинт, недоступный извне, но доступный машине, находящейся в облаке

```
1 user_id = request.args.get("user_id")
2
3 http_client = IHTTPClient()
4 response =
    http_client.Get(f"https://online.sbis.ru/service/get/user/profile/{user_id}")
```

Последствия эксплуатации

На первый взгляд, отправка запросов от сервера может не выглядеть опасной, но фактически это позволяет сделать много нехороших вещей:

1. Просмотр внутренних ресурсов (утечка исходного кода, данных о сотрудниках, информации о серверах и т.д.).
2. Утечка учётных данных, если они передаются в запросе.
3. Обход ограничений при обращении к API. Например при отправке сервисом запросов от лица привилегированного пользователя или сервера.

4. Сканирование внутренней сети.
5. Проникновение во внутреннюю сеть компании через известные уязвимости в стороннем ПО (удалённое исполнение кода в Gitlab, Grafana, Jenkins и т.д.).
6. Вывод ресурсов из строя (Denial-of-service).
7. Чтение файлов через схему file (например, <file:///etc/passwd>)

Как исправить?

Преимущественно, стоит внедрять белые списки для доступа, и проверять относится ли к ним передаваемый адрес. **Важно** - проверки должны выполняться ДО выполнения запроса.

Пример безопасного кода

Можно полностью проверять входной параметр:

```
1 whitelist = ["http://online.sbis.ru/service/ping"]
2
3 url = request.args.get("url")
4
5 if url in whitelist:
6     http_client = IHTTPClient()
7     res = http_client.Get(url)
```

Или, в случае вставки в путь, проверять что передаваемое значение не позволяет выйти из контекста

```
1 def is_valid_uuid(val):
2     try:
3         uuid.UUID(str(val))
4         return True
5     except ValueError:
6         return False
7
8 user_id = request.args.get('user_id')
9
10 if is_valid_uuid(user_id):
11     http_client = IHTTPClient()
12     response =
    http_client.Get(f"https://online.sbis.ru/service/get/user/profile/{user_id}")
```

Альтернативные способы

Если внедрение белых списков невозможно, то можно:

1. Исключить обращение ко внутренним ресурсам (как по доменным именам, так и по ip-адресу).
2. Если происходит обращение ко внешним ресурсам, необходимо исключить утечку служебной информации - заголовки, данные, и т.п.

Полезные ссылки

1. [Шпаргалка об устранении SSRF от OWASP](#)
2. [Статья со множеством способов обхода blacklist и почему он не работает](#)
3. [Статья про SSRF с хабра на русском](#)